

Multi-Camera Streaming over NVR and Raspberry Pi

How office CCTV cameras reach any browser, anywhere — and how the same live feed is reused by other systems without ever overloading the recorder.

Video source	Network Video Recorder (multi-channel RTSP)
Streaming host	Raspberry Pi 5 · 2 GB · 64-bit
Transport	MediaMTX · WebRTC · Cloudflare Tunnel
Public access	HTTPS, password-gated, no port forwarding

01 The problem

Why CCTV is not simply viewable from the internet

An IP camera speaks **RTSP**, a protocol no web browser can play. It also lives on a private LAN address such as **192.168.1.x**, which nothing outside the office can reach. Two separate problems, and both must be solved before anyone can watch a camera from a phone.

Browsers cannot play RTSP

The stream must be repackaged into something a browser understands — here, WebRTC — without re-recording it.

The camera is behind NAT

No cloud service can dial into **192.168.1.x**. Something on the LAN has to originate the connection outward.

The conventional fix is port forwarding: open a hole in the office firewall and point it at the recorder. That exposes the NVR's login page to the whole internet, and CCTV recorders are a well-known target. This design avoids it entirely.

A third problem: the connection limit

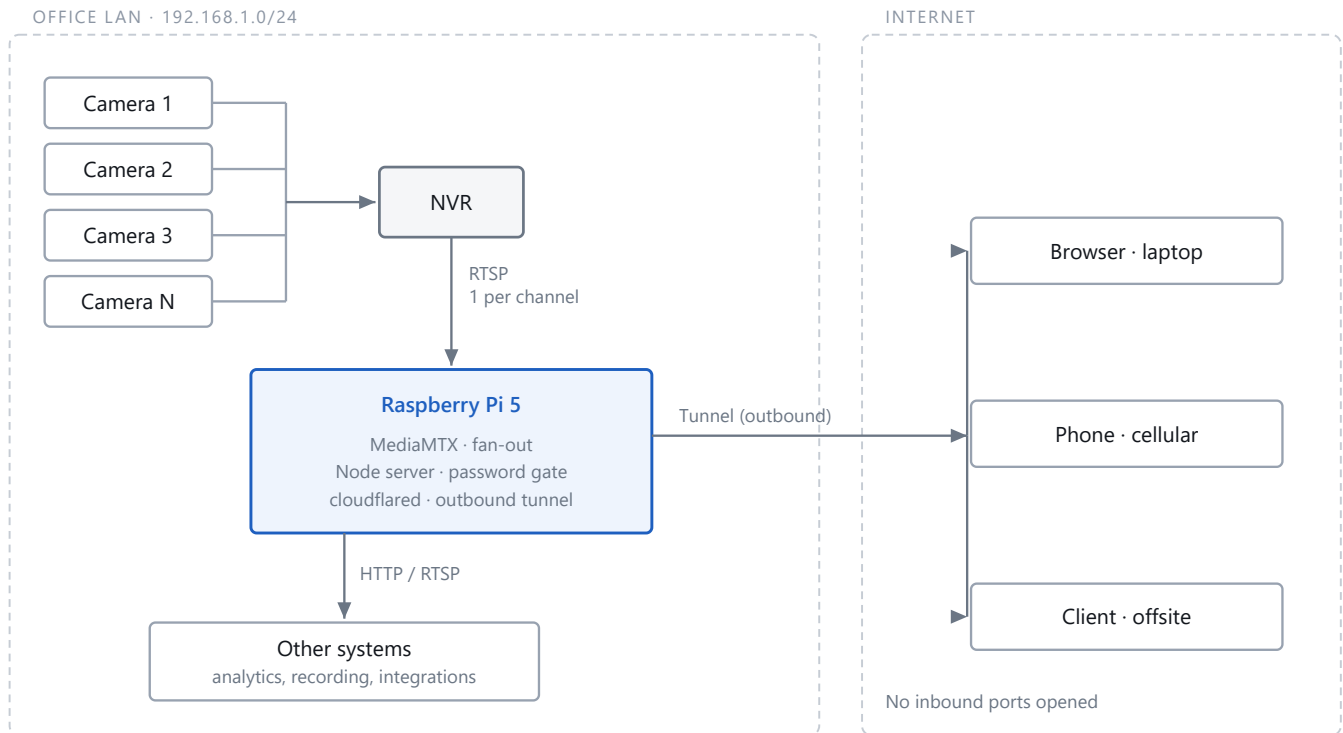
An NVR tolerates only a handful of simultaneous RTSP clients — typically **four to eight in total**, regardless of how many cameras it records. Point a viewer, a phone, and an analytics tool at it directly and the recorder starts refusing connections. Any design that lets consumers talk to the NVR directly will not scale.

THE REQUIREMENT

Reach every camera from anywhere, over HTTPS, with no inbound firewall rule — while holding **exactly one** connection per channel to the recorder no matter how many people or systems are watching.

02 The architecture

One inbound connection per camera, unlimited outbound consumers



The Pi is the only device that talks to the NVR. Everything else talks to the Pi.

Why the Pi sits in the middle

It is the **fan-out point**. The Pi opens one RTSP connection per camera channel and holds it. Every viewer, phone, and downstream system connects to the Pi instead of the recorder. The NVR's connection limit is therefore never a factor: from its point of view there is a single client, permanently attached, no matter how many people are watching.

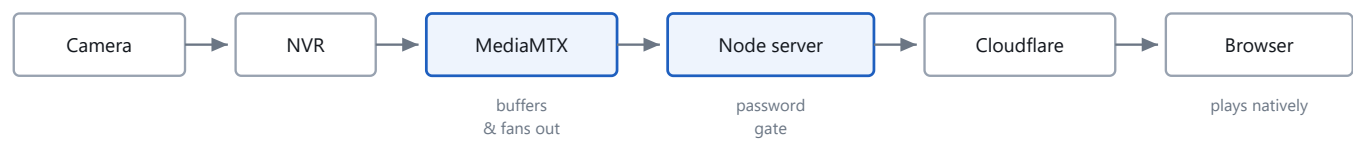
It also originates the tunnel. `cloudflared` dials *out* to Cloudflare and keeps that connection open; public traffic arrives back down the same pipe. No inbound firewall rule, no port forward, no static IP from the ISP, and nothing on the office network is exposed to the internet.

SECURITY CONSEQUENCE

The NVR is never reachable from outside. Only the Pi is, only over HTTPS, and only behind a password. Compromising the public URL yields a video feed — not administrative access to the recorder.

03 How a frame travels

From camera lens to browser window



Stage	What happens
Camera	Encodes H.264 or H.265 and publishes over RTSP to the recorder.
NVR	Records to disk and re-serves each camera as an RTSP channel on one IP address.
MediaMTX	Pulls each channel once, holds it, and re-publishes it simultaneously as WebRTC, RTSP and HTTP. This is the fan-out.
Node server	Enforces the password, serves the viewer page, and proxies WebRTC. MediaMTX is bound to localhost so the gate cannot be bypassed.
Cloudflare	Terminates HTTPS and relays traffic down the tunnel the Pi opened outbound.
Browser	Plays WebRTC natively — no plugin, no app, sub-second latency.

Why WebRTC rather than HLS

HLS — the format used by streaming platforms — chops video into files a few seconds long. That buffering puts the picture **6 to 10 seconds behind real time**. Acceptable for television, poor for security, where the operator needs to see what is happening now.

Approach	Latency	Verdict
HLS	6–10 s	Too far behind for live monitoring
WebRTC	< 1 s	Used here — effectively real time

Passthrough versus transcoding

Where a camera already emits browser-compatible H.264, its packets are copied through untouched — negligible CPU, and many cameras run comfortably at once. Some cameras instead emit full-range `yuvj420p`, which browsers silently refuse to decode: the connection succeeds, data flows, and the viewer sees a grey rectangle. Those channels must be re-encoded.

DESIGN CONSTRAINT

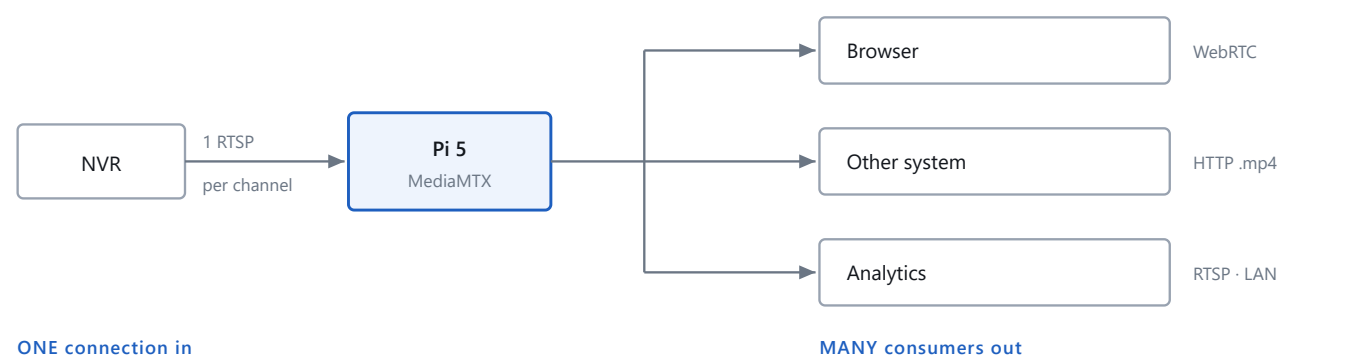
The Raspberry Pi 5 has hardware video *decode* but **no hardware encoder**. Transcoding therefore runs in software and is expensive — roughly two to three channels before the CPU saturates. Passthrough is the intended mode; transcoding is the exception, enabled per camera only where a channel demands it.

04 One feed, many systems

The same live stream, offered in whichever protocol a consumer needs

Because MediaMTX re-publishes each camera in several formats at once, a single inbound stream serves consumers with quite different requirements — a browser, an analytics process, a recording tool — with no additional load on the NVR.

Endpoint	Best suited to	Latency
<code>/?cam=NAME</code>	Human viewers in a browser (WebRTC)	< 1 s
<code>/NAME.mp4</code>	Any system that accepts an HTTP video URL	1–3 s
<code>rtsp://PI:8554/NAME</code>	LAN tools — OpenCV, FFmpeg, recorders	< 1 s



The recorder's connection limit constrains the left side only.

Two honest caveats

The MP4 endpoint is a live pipe, not a file. It streams indefinitely from the current moment. Consumers expecting to seek, scrub, or read a duration will not find those — it has no end. For live viewing and for feeding a detector it is exactly right.

Each MP4 consumer costs a process. Every connection to `/NAME.mp4` spawns its own FFmpeg on the Pi to repackage the stream. The NVR is still protected — the fan-out is real — but the endpoint is not free on the Pi itself. A handful of consumers is fine; dozens is not. Browser viewers on WebRTC do not carry this cost and are the cheaper path where a person, rather than a machine, is watching.

ACCESS CONTROL

Machines cannot complete a browser password prompt, so the MP4 endpoints authenticate with a shared key appended to the URL. If that key is not configured the endpoints are **disabled outright** rather than left open — a camera feed should never become public because a setting was forgotten.

05 Adding cameras

One registry entry per NVR channel

Cameras are not hardcoded. Each is a record in a small database holding its name, display label, RTSP URL, and whether it needs transcoding. The streaming server reads this registry at startup and configures itself accordingly.

An NVR changes almost nothing about this. Rather than a different IP per camera, every channel shares the recorder's address and differs only in the path:

```
# Direct to camera – a different IP per device
rtsp://user:pass@192.168.1.50:554/stream

# Via NVR – one IP, the channel number varies
rtsp://user:pass@192.168.1.10:554/...channel=1
rtsp://user:pass@192.168.1.10:554/...channel=2
```

RTSP paths differ by manufacturer

There is no universal RTSP path. Each vendor uses its own scheme, and identifying the right one is the single most common point of failure during installation.

Manufacturer	Path for channel <i>N</i>
Hikvision	/Streaming/Channels/ <i>N</i> 01 — ch1 = 101, ch2 = 201
Dahua	/cam/realmonitor?channel= <i>N</i> &subtype=0
Uniview	/unicast/cN/s0/live
Reolink	/h264Preview_0 <i>N</i> _main

SUBSTREAMS

Most recorders publish a second, lower-resolution copy of every channel — `subtype=1` on Dahua, `N02` on Hikvision. Typically 640×480 at a fraction of the bitrate. Worth using for any camera that is only being monitored rather than inspected in detail; it cuts bandwidth substantially at no real cost.

06 Deployment sequence

Six stages, in dependency order

1 Prepare the Pi

Install 64-bit Raspberry Pi OS Lite, plus FFmpeg and Node.js. Assign a fixed LAN address so downstream systems have a stable target. The 64-bit build matters — the streaming binary is published for `arm64`, and a 32-bit installation will not find a matching release.

2 Identify the NVR's RTSP scheme

Locate the recorder on the network, confirm RTSP is enabled in its settings — some ship with it switched off — and establish which path scheme it uses by testing one channel. Do this before anything else: every later stage assumes a working URL.

Create a **dedicated read-only account** on the NVR for the Pi rather than reusing the administrator credentials.

3 Install the streaming server

Clone the application, install dependencies, and fetch the MediaMTX binary. Configure the registry connection, the viewer password, and the stream key that enables machine access.

4 Register the channels

Add one entry per camera, in passthrough mode unless a channel proves to need re-encoding. Verify each is reachable before moving on.

5 Establish the tunnel

Start the streaming server and the tunnel. A named tunnel bound to your own domain gives a permanent address; the default produces a new random URL on every restart, which is unsuitable for anything but testing.

6 Make it survive reboots

Register both processes as system services so the Pi resumes streaming automatically after a power cut — the deployment is unattended, and a site visit to restart it defeats the purpose.

KNOWN ISSUE BEFORE LINUX DEPLOYMENT

The streaming server currently resolves the MediaMTX binary by its Windows filename. On Linux that file has no extension, so startup fails with a *"not found"* error even after installation succeeds. A one-line change to select the filename per platform is required before the Pi will run.

07 Capacity and limits

What the hardware supports, and what constrains it

Passthrough channels

8–16

Limited by network bandwidth, not CPU. Packets are copied, not processed.

Transcoded channels

2–3

Software encoding only — the Pi 5 has no hardware encoder. Avoid where possible.

Bandwidth is the real ceiling

In passthrough the Pi computes very little; it moves data. Budget roughly **4 Mbit/s per 1080p stream**, and note that each concurrent viewer consumes another copy — six cameras watched by five people is around 120 Mbit/s. The Pi 5's gigabit interface absorbs that comfortably.

WHY THE HARDWARE GENERATION MATTERS

The Raspberry Pi 3B+ shares its 100 Mbit network interface with the USB bus, capping realistic throughput near 8–10 concurrent 1080p streams before the interface saturates. The Pi 5 provides true dedicated gigabit — roughly a tenfold increase in headroom, and the reason this design is viable on it.

Deliberate exclusion: on-device video analytics

Object detection and face recognition are **out of scope for this deployment**. The supporting libraries require several gigabytes of memory to load, which a 2 GB Pi cannot provide, and inference on its CPU would run at a small fraction of real time.

The architecture accommodates it regardless: analytics runs on a separate, more capable machine and consumes the Pi's existing RTSP or MP4 endpoint like any other client. Nothing about the streaming layer changes when it is added, and the NVR sees no additional load.

08 Summary

The design in eight points

Property	Outcome
Access	Any browser, anywhere, over HTTPS. No app, no plugin, no VPN client.
Latency	Under one second via WebRTC — effectively live.
Firewall	No inbound rules. The tunnel is opened outbound by the Pi.
NVR load	One connection per channel, constant, regardless of viewer count.
Reuse	The same feed is offered as WebRTC, HTTP and RTSP concurrently.
Security	Password-gated; the recorder is never exposed to the internet.
Running cost	Tunnel bandwidth is unmetered; the Pi draws a few watts.
Scaling	Add a camera by adding a registry entry — no code changes.

Prerequisites before installation

- NVR manufacturer and model, and the number of channels to publish.
- A dedicated read-only NVR account for the Pi.
- RTSP confirmed enabled on the recorder.
- A fixed LAN address for the Pi.
- A domain on Cloudflare, if a permanent public URL is required.

Outstanding engineering items

- **Platform fix** — resolve the MediaMTX binary name per operating system, so the server starts on Linux.
- **Per-channel verification** — probe each NVR channel to determine whether it can pass through untouched or requires re-encoding.
- **Service definitions** — register the server and tunnel as system services for unattended restart.